

ESP32 Snake Game with RGB LED Matrix

Project Documentation

Author: xondre16

1 Introduction

This project implements the classic Snake game on an ESP32-based Wemos D1 R32 board using an Analog RGB LED Shield as the display. The game is rendered on a 16×8 RGB LED matrix and controlled using four push-buttons integrated on the shield.

The application is written in C using the ESP-IDF framework and developed with PlatformIO in Visual Studio Code. The project demonstrates embedded system design, real-time control, and multiplexed LED driving.

2 Hardware Overview

2.1 Components

- **Microcontroller:** ESP32 (Wemos D1 R32)
- **Display:** Analog RGB LED Shield (16×8 RGB LED matrix)
- **Column decoder:** 74HCT154
- **LED driver:** TLC5947
- **Input:** Four push-buttons on the shield

2.2 Display Control Principle

The RGB LED matrix consists of 16 columns and 8 rows, with each pixel implemented as an RGB LED. Due to hardware constraints, the display is controlled using required time multiplexing, where only one column is active at a time.

Column selection is handled by the 74HCT154 decoder using four GPIO address lines. The decoder outputs are active-low and control MOSFETs that supply power to individual columns.

LED brightness and color are controlled by the TLC5947 PWM driver, which provides 24 channels corresponding to 8 RGB LEDs per column. Each channel supports 12-bit resolution. For each column refresh, a 288-bit data frame is transmitted via SPI.

2.3 Buttons

The four push-buttons on the RGB LED Shield are connected to ESP32 GPIO pins and configured as inputs with internal pull-up resistors. The button functions are defined in software.

3 Software Design

3.1 Project Structure

- **main.c** – System initialization and task creation
- **gpio_hw.c/h** – GPIO configuration
- **display.c/h** – Frame buffer handling and display refresh
- **colors.c/h** – RGB color definitions
- **snake_game.c/h** – Game logic and state management

3.2 Initialization

At startup, GPIO pins are configured, the SPI interface for the TLC5947 is initialized, and FreeRTOS tasks for display refreshing and game logic are created.

4 Display Refresh

The display is refreshed continuously by scanning all columns:

- Select column using the 74HCT154 decoder
- Transmit PWM data for the active column via SPI
- Latch and display the data for a short time interval

Color values for the entire display are stored in a frame buffer and converted to the TLC5947 format during refresh.

5 Game Logic

5.1 Gameplay Rules

The game operates on a 16×8 grid corresponding to the LED matrix resolution.

- The snake moves automatically at a configurable speed
- Direction changes are controlled by buttons; direct reversal is not allowed
- **Blue food** increases snake length by one segment
- **Red food** decreases snake length by one segment
- The game ends if the snake collides with itself or its length drops below five segments

5.2 Game States

The application includes the following states:

- Active gameplay
- Game over animation
- Screensaver mode
- Start countdown

6 User Interaction

6.1 Controls

- **UP (SW2):** Move up
- **DOWN (SW1):** Move down
- **LEFT (SW3):** Move left
- **RIGHT (SW4):** Move right

6.2 Screensaver Mode

In screensaver mode:

- **UP / DOWN:** Adjust snake speed
- **RIGHT:** Change snake color
- **LEFT:** Restart the game

7 Build and Execution

1. Install PlatformIO with ESP-IDF support
2. Connect the board via USB
3. Configure the serial port in `platformio.ini`
4. Build and upload the project using PlatformIO

8 Testing

Basic functionality was verified by testing individual LEDs, button inputs, and game behavior. Serial logging was used for debugging and validation.

9 Demonstration of functionality

Video: <https://youtu.be/vj9CI7hBAVI>

10 Conclusion

The project implements a functional Snake game on an ESP32 using a multiplexed RGB LED matrix. The solution combines efficient hardware control with structured software design and fulfills all project requirements.